03. 평가 과정 (Evaluation Process)

개요

평가 과정은 Monkey 언어의 인터프리터 모드에서 파서가 생성한 AST(Abstract Syntax Tree)를 실제로 실행하여 결과를 얻는 단계입니다. 이 과정에서 `Eval()` 함수가 핵심 역할을 하며, 각 AST 노드 타입에 따라 적절한 평가 로직을 수행합니다.

1. Eval() 함수와 객체 시스템

1.1 Eval() 함수의 핵심 구조

`Eval()` 함수는 `evaluator/evaluator.go` 에 정의되어 있으며, AST 노드와 환경(Environment)을 받아서 객체(Object)를 반환하는 재귀적 함수입니다.

```
func Eval(node ast.Node, env *object.Environment) object.Object {
switch node := node.(type) {
case *ast.Program:
return evalProgram(node, env)
case *ast.ExpressionStatement:
return Eval(node.Expression, env)
case *ast.BlockStatement:
return evalBlockStatement(node, env)
case *ast.ReturnStatement:
val := Eval(node.ReturnValue, env)
```

```
if isError(val) {
return val
}
return &object.ReturnValue{Value: val}
case *ast.LetStatement:
val := Eval(node.Value, env)
if isError(val) {
return val
}
env.Set(node.Name.Value, val)
// ... 더 많은 케이스들
}
return nil
}
```

1.2 Switch 문을 통한 노드 타입별 처리

`Eval()` 함수는 타입 스위치(type switch)를 사용하여 각 AST 노드 타입에 따라 다른 평가 로직을 수행합니다:

1.2.1 프로그램과 문장 처리

- **`*ast.Program`** : 전체 프로그램을 평가하는 `evalProgram()` 호출
- **`*ast.ExpressionStatement`** : 표현식 문장의 표현식 부분을 재귀적으로 평가
- **`*ast.BlockStatement`** : 블록 내의 모든 문장을 순차적으로 평가하는 `evalBlockStatement()` 호출

- **`*ast.ReturnStatement`** : 반환값을 평가하고 `ReturnValue` 객체로 래핑
- **`*ast.LetStatement`** : 변수 할당문 처리, 값을 평가한 후 환경에 바인딩

1.2.2 표현식 처리

- **`*ast.IfExpression`** : 조건문 평가를 위한 `evalIfExpression()` 호출
- **`*ast.PrefixExpression`** : 전위 연산자 처리 (`!`, `-` 등)
- **`*ast.InfixExpression`** : 중위 연산자 처리 (`+`, `-`, `==` 등)
- **`*ast.CallExpression`** : 함수 호출 처리
- **`*ast.IndexExpression`** : 배열/해시 인덱스 접근 처리

1.2.3 리터럴과 식별자 처리

- **`*ast.Identifier`** : 변수 이름 해결을 위한 `evalIdentifier()` 호출
- **`*ast.FunctionLiteral`** : 함수 리터럴을 `Function` 객체로 변환
- **`*ast.IntegerLiteral`** : 정수 리터럴을 `Integer` 객체로 변환
- **`*ast.StringLiteral`** : 문자열 리터럴을 `String` 객체로 변환
- **`*ast.Boolean`** : 불린 리터럴을 `Boolean` 객체로 변환
- **`*ast.ArrayLiteral`** : 배열 리터럴을 `Array` 객체로 변환
- **`*ast.HashLiteral`** : 해시 리터럴을 `Hash` 객체로 변환

1.3 에러 처리 메커니즘

평가 과정에서 발생하는 에러는 `Error` 객체로 표현되며, 각 평가 단계에서 에러 검사를 수행합니다:

```
// 에러 검사 예시
val := Eval(node.ReturnValue, env)
if isError(val) {
```

```
return val // 에러가 발생하면 즉시 반환
}
```

`isError()` 함수는 객체가 에러 타입인지 확인하고, 에러가 발생하면 평가를 중단하고 에러를 상위로 전파합니다.

1.4 재귀적 평가 구조

`Eval()` 함수는 재귀적으로 동작하여 복잡한 표현식을 단계별로 평가합니다:

```
// 중위 표현식 평가 예시
case *ast.InfixExpression:
left := Eval(node.Left, env) // 왼쪽 피연산자 평가
if isError(left) {
return left
}
right := Eval(node.Right, env) // 오른쪽 피연산자 평가
if isError(right) {
return right
}
return evalInfixExpression(node.Operator, left, right) // 연산 수행
```

이러한 재귀적 구조를 통해 `1 + 2 * 3` 과 같은 복잡한 표현식도 올바른 순서로 평가됩니다.

2. 객체 타입과 Object 인터페이스

2.1 Object 인터페이스의 구조

Monkey 언어의 모든 값은 `Object` 인터페이스를 구현하는 객체로 표현됩니다. 이 인터페이스는 `object/object.go` 에 정의되어 있습니다:

```
type Object interface {
Type() ObjectType
Inspect() string
}
```

- **`Type()`** : 객체의 타입을 반환하는 메서드
- **`Inspect()`** : 객체의 문자열 표현을 반환하는 메서드 (디버깅과 출력에 사용)

2.2 ObjectType 상수 정의

각 객체 타입은 문자열 상수로 정의되어 있습니다:

```
type ObjectType string

const (
IntegerObj = "Integer"
BooleanObj = "Boolean"
NullObj = "Null"
ReturnValueObj = "ReturnValue"
ErrorObj = "Error"
FunctionObj = "Function"
StringObj = "String"
BuiltinObj = "Builtin"
```

```
ArrayObj = "Array"

HashObj = "Hash"

CompiledFunctionObj = "CompiledFunction"

ClosureObj = "Closure"

)
```

2.3 기본 데이터 타입 객체들

2.3.1 Integer 객체

정수 값을 표현하는 객체입니다:

```
type Integer struct {

Value  int64

}


func (i *Integer) Type() ObjectType { return  IntegerObj }

func (i *Integer) Inspect() string { return  fmt.Sprintf("%d", i.Value) }

func (i *Integer) HashKey() HashKey {

return HashKey{Type: i.Type(), Value: uint64(i.Value)}

}
```

- `HashKey()` 메서드를 구현하여 해시 테이블의 키로 사용 가능
- `int64` 타입으로 큰 정수도 처리 가능

2.3.2 Boolean 객체

불린 값을 표현하는 객체입니다:

```go
type Boolean struct {
Value  bool
}

func (b *Boolean) Type() ObjectType { return  BooleanObj }
func (b *Boolean) Inspect() string { return  fmt.Sprintf("%t", b.Value) }
func (b *Boolean) HashKey() HashKey {
var  value  uint64
if  b.Value {
value = 1
} else {
value = 0
}
return HashKey{Type: b.Type(), Value: value}
}
```

- 해시 키로 사용할 때 `true` 는 1, `false` 는 0으로 변환

2.3.3 String 객체

문자열 값을 표현하는 객체입니다:

```go
type String struct {
Value  string
```

```go
}


func (s *String) Type() ObjectType { return  StringObj }
func (s *String) Inspect() string { return  s.Value }
func (s *String) HashKey() HashKey {
h := fnv.New64a()
h.Write([]byte(s.Value))
return HashKey{Type: s.Type(), Value: h.Sum64()}
}
```

- FNV 해시 알고리즘을 사용하여 문자열의 해시 값 생성

2.3.4 Null 객체

null 값을 표현하는 특별한 객체입니다:

```go
type Null struct{}


func (n *Null) Type() ObjectType { return  NullObj }
func (n *Null) Inspect() string { return  "null" }
```

- 값이 없음을 나타내는 싱글톤 객체

2.4 제어 흐름 관련 객체들

2.4.1 ReturnValue 객체

함수에서 반환되는 값을 래핑하는 객체입니다:

```go
type ReturnValue struct {
Value Object
}

func (rv *ReturnValue) Type() ObjectType { return ReturnValueObj }
func (rv *ReturnValue) Inspect() string { return rv.Value.Inspect() }
```

- 실제 반환값을 `Value` 필드에 저장
- 함수 실행 중 `return` 문을 만났을 때 생성

2.4.2 Error 객체

런타임 에러를 표현하는 객체입니다:

```go
type Error struct {
Message string
}

func (e *Error) Type() ObjectType { return ErrorObj }
func (e *Error) Inspect() string { return "ERROR: " + e.Message }
```

- 에러 메시지를 문자열로 저장
- 평가 과정에서 발생하는 모든 에러를 표현

2.5 함수 관련 객체들

2.5.1 Function 객체

사용자 정의 함수를 표현하는 객체입니다:

```go
type Function struct {
Parameters []*ast.Identifier
Body *ast.BlockStatement
Env *Environment
}

func (f *Function) Type() ObjectType { return FunctionObj }
func (f *Function) Inspect() string {
var out bytes.Buffer
var params []string
for _, p := range f.Parameters {
params = append(params, p.String())
}
out.WriteString("fn")
out.WriteString("(")
out.WriteString(strings.Join(params, ", "))
out.WriteString(") {\n")
out.WriteString(f.Body.String())
out.WriteString("\n}")
return out.String()
}
```

- `Parameters` : 함수의 매개변수 목록
- `Body` : 함수의 본문 (AST 노드)
- `Env` : 함수가 정의된 환경 (클로저 지원)

2.5.2 Builtin 객체

내장 함수를 표현하는 객체입니다:

```
type BuiltinFunction func(args ...Object) Object


type Builtin struct {
Fn BuiltinFunction
}


func (b *Builtin) Type() ObjectType { return  BuiltinObj }
func (b *Builtin) Inspect() string { return  "builtin function" }
```

- Go 함수를 래핑하여 Monkey 언어에서 사용할 수 있게 함
- `len` , `puts` , `first` 등의 내장 함수들이 이 타입으로 구현

2.6 컬렉션 타입 객체들

2.6.1 Array 객체

배열을 표현하는 객체입니다:

```
type Array struct {
Elements []Object
```

```go
}

func (ao *Array) Type() ObjectType { return ArrayObj }
func (ao *Array) Inspect() string {
var out bytes.Buffer
var elements []string
for _, e := range ao.Elements {
elements = append(elements, e.Inspect())
}
out.WriteString("[")
out.WriteString(strings.Join(elements, ", "))
out.WriteString("]")
return out.String()
}
```

- 다양한 타입의 객체들을 요소로 가질 수 있는 동적 배열

2.6.2 Hash 객체

해시 테이블(딕셔너리)을 표현하는 객체입니다:

```go
type HashPair struct {
Key Object
Value Object
}
```

```go
type Hash struct {
Pairs  map[HashKey]HashPair
}


func (h *Hash) Type() ObjectType { return  HashObj }
func (h *Hash) Inspect() string {
var  out bytes.Buffer
var  pairs []string
for  _, pair := range  h.Pairs {
pairs = append(pairs, fmt.Sprintf("%s: %s",
pair.Key.Inspect(), pair.Value.Inspect()))
}
out.WriteString("{")
out.WriteString(strings.Join(pairs, ", "))
out.WriteString("}")
return  out.String()
}
```

- `Hashable` 인터페이스를 구현하는 객체만 키로 사용 가능
- `HashKey` 를 통해 효율적인 해시 테이블 구현

2.7 Inspect() 메서드를 통한 객체 출력

모든 객체는 `Inspect()` 메서드를 통해 사람이 읽을 수 있는 문자열 형태로 변환됩니다:

- **정수**: `42`
- **불린**: `true`, `false`
- **문자열**: `hello world`
- **배열**: `[1, 2, 3]`
- **해시**: `{key: value, name: John}`
- **함수**: `fn(x, y) { x + y }`
- **에러**: `ERROR: identifier not found: x`

이 메서드는 REPL에서 결과를 출력하거나 디버깅할 때 사용됩니다.##

3. Environment와 스코프 관리 시스템

3.1 Environment 구조체의 기본 구조

`Environment` 는 변수와 함수의 바인딩을 관리하는 핵심 구조체입니다. `object/environment.go` 에 정의되어 있습니다:

```
type Environment struct {
store  map[string]Object
outer *Environment
}
```

- **`store`** : 변수명을 키로, 객체를 값으로 하는 해시 맵
- **`outer`** : 외부(상위) 환경에 대한 참조 (스코프 체인 구현)

3.2 Environment 생성 함수들

3.2.1 새로운 환경 생성

```go
func NewEnvironment() *Environment {
s := make(map[string]Object)
return &Environment{store: s}
}
```

- 전역 스코프나 독립적인 환경을 생성할 때 사용
- `outer` 필드는 `nil` 로 설정됨

3.2.2 중첩된 환경 생성

```go
func NewEnclosedEnvironment(outer *Environment) *Environment {
env := NewEnvironment()
env.outer = outer
return env
}
```

- 함수 호출이나 블록 스코프를 위한 새로운 환경 생성
- 외부 환경에 대한 참조를 유지하여 스코프 체인 형성

3.3 변수 바인딩과 해결 메커니즘

3.3.1 변수 설정 (Set)

```go
func (e *Environment) Set(name string, val Object) Object {
e.store[name] = val
return val
```

```
}
```

- 현재 환경에 변수를 바인딩
- 같은 이름의 변수가 있으면 덮어씀 (shadowing)
- 설정된 값을 반환

3.3.2 변수 조회 (Get)

```go
func (e *Environment) Get(name string) (Object, bool) {
obj, ok := e.store[name]
if !ok && e.outer != nil {
obj, ok = e.outer.Get(name)
}
return obj, ok
}
```

변수 조회는 스코프 체인을 따라 수행됩니다:

1. **현재 환경 검색**: 먼저 현재 환경의 `store` 에서 변수를 찾음
2. **상위 환경 검색**: 현재 환경에 없으면 `outer` 환경에서 재귀적으로 검색
3. **결과 반환**: 변수를 찾으면 객체와 `true` 를, 못 찾으면 `nil` 과 `false` 를 반환

3.4 스코프 체인과 변수 해결 과정

3.4.1 스코프 체인의 구조

```
전역 환경 (Global Environment)
```

```
↑
함수 환경 (Function Environment)
↑
블록 환경 (Block Environment)
```

각 환경은 `outer` 포인터를 통해 상위 환경과 연결되어 스코프 체인을 형성합니다.

3.4.2 변수 해결 예시

다음과 같은 코드에서의 변수 해결 과정을 살펴보겠습니다:

```
let x = 10;
let fn = fn(y) {
let z = 20;
x + y + z
};
fn(5);
```

1. **전역 환경**: `x = 10`, `fn = Function객체`
2. **함수 호출 시**: 새로운 환경 생성, `outer` 는 전역 환경을 가리킴
3. **함수 환경**: `y = 5`, `z = 20`
4. **변수 해결**:

- `z` : 현재 함수 환경에서 발견 → `20`
- `y` : 현재 함수 환경에서 발견 → `5`
- `x` : 현재 환경에 없음 → 전역 환경에서 발견 → `10`

3.5 함수 호출 시 새로운 환경 생성

3.5.1 함수 환경 확장

```
func extendedFunctionEnv(fn *object.Function, args []object.Object) *object.Env
env := object.NewEnclosedEnvironment(fn.Env)
for paramIdx, param := range fn.Parameters {
env.Set(param.Value, args[paramIdx])
}
return env
}
```

함수 호출 시 환경 생성 과정:

1. **새 환경 생성**: 함수의 정의 환경(`fn.Env`)을 외부 환경으로 하는 새 환경 생성
2. **매개변수 바인딩**: 함수의 매개변수들을 전달받은 인수들과 바인딩
3. **클로저 지원**: 함수가 정의된 환경을 기억하여 클로저 기능 제공

3.5.2 클로저 동작 예시

```
let makeCounter = fn() {
let count = 0;
fn() {
count = count + 1;
count
}
};
```

```
let counter = makeCounter();

counter(); // 1

counter(); // 2
```

이 예시에서:

1. `makeCounter` 호출 시 `count = 0` 인 환경 생성
2. 내부 함수는 이 환경을 `outer` 로 참조하는 새 환경에서 실행
3. `count` 변수는 외부 환경에서 찾아서 수정됨

3.6 스코프와 변수 섀도잉

3.6.1 변수 섀도잉 (Variable Shadowing)

```
let x = 10;

let fn = fn(x) { // 매개변수 x가 전역 x를 가림

x + 1

};

fn(5); // 결과: 6 (전역 x=10이 아닌 매개변수 x=5 사용)
```

3.6.2 블록 스코프 (향후 확장 가능)

현재 구현에서는 함수 스코프만 지원하지만, 환경 시스템은 블록 스코프도 쉽게 지원할 수 있도록 설계되었습니다:

```
// 향후 지원 가능한 블록 스코프

let x = 10;
```

```
{
let x = 20; // 블록 내에서 x 섀도잉
x // 20
}
x // 10
```

3.7 환경과 가비지 컬렉션

Go의 가비지 컬렉터가 자동으로 처리하지만, 환경 구조의 메모리 관리 특성:

- **참조 체인**: 내부 환경이 외부 환경을 참조하므로 외부 환경이 메모리에 유지됨
- **클로저 메모리**: 클로저가 외부 변수를 참조하는 동안 해당 환경이 메모리에 유지됨
- **자동 해제**: 더 이상 참조되지 않는 환경은 자동으로 가비지 컬렉션됨

이러한 환경 시스템을 통해 Monkey 언어는 렉시컬 스코핑(lexical scoping)과 클로저를 완전히 지원합니다.